

People leave the chain by changing the money rules. That has happened many times, and each time it is a new coin. Nodes stay equal even though the people who write the code never will, and that still does not entitle those people to a throne.

The model works

Benevolent dictatorship produces good software. Linux runs most of the internet's infrastructure and was governed for decades by one person's judgment about what merged. Python, Django, and Rust all had stretches of single-authority governance, and the work held up.

Concentrated authority makes architectural decisions faster than committees do. It keeps one design across a large codebase, and it ends fights that would otherwise eat years. Linux is a product, and a small group decides what goes in the kernel while everyone else downloads the result. Someone has to pick what ships, and a hierarchy is the right way to run that.

Forking the kernel is allowed. Almost nobody does it. The fork is yesterday's code. The patches people want keep going into the original project. Bitcoin node operators do the same thing. They download Core and wait for the next release. Bitcoin's money rules and the extra services people want on top of them are not the same product. Treating a wallet fight or a relay fight as if it were a change to the money is how those fights become governance crises.

The pattern is right for ordinary software and wrong for money.

The structure arrives on its own

The hierarchy also shows up because paid engineers bring their workplace with them.

Most of the work is done by people employed to do it. The majority of Linux kernel contribution comes from paid engineers at Intel, Red Hat, Google, IBM, and a handful of others, and that has been true for well over a decade. The same pattern shows up across most infrastructure projects of any size. Volunteer contribution is real, yet paid engineers carry the load.

People who spend forty hours a week inside a company's engineering process bring that process with them. Review turns into an approval chain. Roadmaps get set by a small group. Release management belongs to specific people on a schedule. Disputes go up to whoever is supposed to end them. Nobody votes any of that in. It arrives as professional habit, because it is the only way of coordinating software most working engineers have ever practiced.

Core's contributors are largely employed or funded by a small number of organizations. The governance system that emerged at Core is what happens when people trained in corporate engineering coordinate a project and nobody is asked to design those rules on purpose.

Coin Dance. That was a policy protest on the same codebase, still one client. It did not give anyone a place to stay. Luke's Knots later left Bitcoin on a BLAKE2b proof-of-work hard fork. [Making Core Irrelevant](#) treats the reachable-node split as demand for a policy alternative, not as a break in the implementation monoculture." "Running different defaults is cheap compared to risking a chain split. Miners can refuse an upgrade, so a consensus change can be blocked. That veto is rarely used. The cost of refusing a default falls on the operator. The benefit of setting it accrues to whoever sets it. Nobody has to capture consensus enforcement to govern this way." "What Core actually looks like" "The stated model is rough consensus among a broad contributor base. In the merge log, a handful of accounts do most of the merges. Bitcoin Core is not formally a benevolent dictatorship and has never claimed to be, yet how it actually merges looks like a dictatorship with the name taken off, which is worse, because undeclared concentration is harder to see and harder to contest." "SECURESOVEREIGN The Vertical Layer Problem 4 / 9"

What a horizontal development layer looks like

Vote weight comes from work recorded in public, and a title does not confer it.

The [Bitcoin Commons white paper](#) does that with two questions at once. One is where the change lives, in the consensus specification, the protocol, the node, or an optional module. The other is what kind of change it is, a routine fix, a feature, something that touches consensus validation, an emergency patch, or a change to the governance rules themselves. When both apply, you take the higher signature count and the longer wait. A bug fix in the consensus layer still needs the consensus-layer bar, while a feature in an optional module does not.

Routine maintenance clears on fewer signatures and a short wait. Consensus-adjacent changes need near-unanimity and a delay measured in months. Changes to the governance rules themselves need more than that. A contributor cannot approve their own merge, and even a repository administrator cannot bypass the signature check.

Optional features load as process-isolated modules outside the money rules. Lightning, mining interfaces, and similar tools can compete without anyone treating them as a consensus change. The base node still validates blocks against the specification, so a feature fight does not become a throne fight.

A further ladder, vote weight on the crates you have actually shipped, is specified as a complement to that merge policy, and it is not what is enforced today.

Every action is cryptographically signed and publicly auditable. Power is visible because anyone can check the record, not because people are supposed to behave. The software layer cannot make every contributor equal the way every node is equal, and it can make the record checkable, which is what matters for who may change the code.

Where it fails

The crate ladder, if it ever ships, can let a shallow contributor outvote the person who wrote the subsystem. A single merged pull request can put someone on questions they cannot evaluate. Someone who has shown they understand the architecture still has to gate who votes. That limit is real. Below it, more votes make worse decisions.

The merge policy specified today has a different failure, because the people who hold the keys can collude. The keys are supposed to sit in different places for that reason, and a governance ruleset has to be forkable without splitting the money.

Distribution is slow when you need a same-day security patch. Ordinary merge rules should stay slow. The specified answer is a declared emergency with a clock, which is the substitute for a person who can merge anything.

If one implementation still defines the rules everyone else must copy, distributed governance is theater. A specification plus full-chain tests is what turns "we match Core" from a claim into a pass/fail check, and that only works if the specification matches mainnet, including the ugly historical bugs.

Commons has not run this merge policy at scale. Cryptographic merge authorization is designed and stays off until security review, key management, and community validation. Gates and declared emergencies are rules on a public record, and a person constrained only by custom is what they replace. That should break differently under pressure, and it remains a prediction until someone runs it. The specification, the emergency path, and cheap exit have to exist because of those limits.

The governance system described is [Bitcoin Commons](#), the specification is the [Orange Paper](#), and the implementation is [BLVM](#).

Without a written specification, fights still go to whoever has been around longest. There is nothing independent to measure a judgment against, so authority reconcentrates no matter what a governance document says. Core cannot be repaired by redistributing merge rights, and the missing specification is what keeps the monopoly in place, because that missing document is doing work. The longer write-up is [Who Controls Bitcoin](#).

A written specification gives contributors something to be right about that is not a person's opinion, and writing it is the hard part, because Bitcoin's live rules include serialization quirks, historical bugs the network accepted, and a policy and consensus boundary that is blurrier in practice than on paper. A specification that matches mainnet has to encode all of it, including the parts nobody would design on purpose, which makes it a large contested document that nobody writes in a week.

With a specification, fights become what the document says. Formal verification and machine-checkable proofs shrink the room to argue about what the specification means.

Exit has to be cheap enough to be credible. A formally specified implementation can be forked against the specification without reverse-engineering Core. A governance ruleset can be exported as a signed package and replaced without splitting the ledger, which is what makes a fork usable. Cheap exit keeps people honest in ways internal process cannot. The sequencing of that exit is [Making Core Irrelevant](#).

The standard

Bitcoin does not get a pass because some of the people writing the code are skilled or well-intentioned. Failed monetary institutions had skilled, well-intentioned people too. What matters is whether someone with merge authority still decides what Bitcoin nodes run on ordinary work, and whether an inflation bug can be patched without giving that person the power to merge whatever they want.

Commons specifies the second as a three-class emergency system. Emergency keyholders declare a class. That declaration changes the merge rules until a clock expires. Network-threatening work gets the fastest path, including no review wait while the declaration is live. Less severe classes keep longer waits and higher bars. Declaring an emergency is a different act from tagging a pull request as emergency work. After the clock, a written post-mortem is required and ordinary rules return. Nobody keeps a low bar because they might need one later.

The overnight patch then goes through a declared class and a clock. A governance system that works only while the right person is in charge was built for Linux, and it does not belong on Bitcoin's node software.

Sources

- [Bitcoin Governance Research](#). Merge/review concentration, Gini, self-merge
- [Brink, Engineering Impact Report 2025](#). One-person merge share
- [Coin Dance](#). Knots reachable-node share
- [Bitcoin Commons white paper](#). Dual bars (where and what), optional modules, three-class emergency (declaration vs PR tag), governance-ruleset exit
- [Orange Paper / Bitcoin Commons consensus spec](#)
- [Bitcoin Commons documentation](#). BLVM Specification Lock; cryptographic merge authorization designed, off

SECURESOVEREIGN

Markdown is the canonical source: <https://secsov.com/articles/the-vertical-layer-problem/index.md>

Formatted snapshot of <https://secsov.com/articles/the-vertical-layer-problem>

Josh · Secure Sovereign